Day 4 – Doing Scrum – Sprint Simulation

Topics covered: Sprint Planning · Daily Stand-up · Backlog Refinement · Sprint Review and Retrospective · Agile and DevOps Culture · Distributed Teams

Set 1 – Digital Wallet Application – Mid-Sprint Change

Prepared by: N.Moksha Sai Programme: Hexaware Training Day: 4

Business Scenario	A fintech team of five developers is in a 10-day sprint. Previous sprint velocities were 21, 24 and 23 story points. On day 4 the business asks for a refund-to-wallet feature.
Team and assumptions	Developers: Anil, Priya, Karthik, Meera, Sana. Roles: Product Owner (PO), Scrum Master (SM). Focus capacity of 6 productive hours per developer per day (the rest goes to ceremonies, reviews and support). Story points burn down only when a story fully meets the Definition of Done.

Task 1 – Sprint Planning: velocity, stories and Sprint Goal

Average velocity = (21 + 24 + 23) ÷ 3 = 68 ÷ 3 = 22.67, rounded to **23 story points**. This is the forecast capacity for the sprint.

Sprint 1	Sprint 2	Sprint 3	Average velocity
21	24	23	22.67 ≈ 23

The PO's ordered product backlog is pulled from the top until the 23-point capacity is reached. WAL-104 (3 pts) would take the sprint to 26, so it stays in the backlog.

ID	User story	Points	Sprint 1 decision
WAL-101	As a new user, I can register and complete KYC so that I can open a wallet	5	✔️ Selected
WAL-102	As a user, I can add money to my wallet through UPI	5	✔️ Selected
WAL-103	As a user, I can transfer money to another wallet user	8	✔️ Selected
WAL-105	As a user, I can log in with fingerprint / face authentication	5	✔️ Selected
WAL-104	As a user, I can view and filter my transaction history	3	–
WAL-106	As a user, I get a low-balance alert	2	–
WAL-109	As a user, I can download a PDF statement	3	–
WAL-107	As a user, I can see a spending analytics dashboard	8	–
WAL-108	As a user, I can pay a merchant by scanning a QR code	8	–
WAL-110	As a user, I can earn referral rewards	5	–
	Total selected for the sprint	**23**	**= average velocity**

> **Sprint Goal:** By the end of Day 10, a KYC-verified user can add money to the wallet through UPI and transfer it to another wallet user (95% of transfers complete in under 2 seconds), with every selected story meeting the Definition of Done and a demo-ready build on the staging environment.

Task 2 – Stories broken into tasks with hour estimates

The three largest selected stories are broken into tasks small enough to finish in one to two days.

WAL-101 – Registration and KYC (5 pts)

Task	Description	Estimate (h)
T1.1	Design sign-up / KYC API contract and database schema	4
T1.2	Build registration API with OTP verification	8
T1.3	KYC document upload and third-party verification integration	10
T1.4	Sign-up and KYC screens	8
T1.5	Unit and integration tests	6
T1.6	Code review and fixes	3
	Total	**39**

WAL-102 – Add money through UPI (5 pts)

Task	Description	Estimate (h)
T2.1	Payment gateway sandbox setup	4
T2.2	Top-up API and wallet ledger entry	8
T2.3	Webhook handling for payment success / failure	6
T2.4	Add-money screens	6
T2.5	Idempotency and retry handling	4
T2.6	Unit and integration tests	6
T2.7	Code review and fixes	3
	Total	**37**

WAL-103 – Wallet-to-wallet transfer (8 pts)

Task	Description	Estimate (h)
T3.1	Transfer API with atomic debit and credit	12
T3.2	Balance check and transfer-limit validation	6
T3.3	Velocity / fraud rule for repeated transfers	6
T3.4	Transfer and confirmation screens	8
T3.5	Sender and receiver notifications	4
T3.6	Concurrency and load tests	8
T3.7	Unit tests	6
T3.8	Code review and fixes	4
T3.9	API documentation	2
	Total	**56**

Capacity check: 5 developers × 10 days × 6 focus hours = 300 h. The three broken-down stories need 39 + 37 + 56 = 132 h (44% of capacity), which leaves room for WAL-105, ceremonies and defects.

Sprint board – snapshot at the Day 3 morning stand-up

To Do	In Progress	Done
T1.6 Review and fixes (3h) T2.4 Add-money screens (6h) T2.5 Idempotency / retry (4h) T2.6 Tests (6h) T2.7 Review (3h) T3.2–T3.9 (46h) All WAL-105 tasks	T1.5 Tests (6h) – Priya T2.3 Webhook handling (6h) – Karthik T3.1 Transfer API (12h) – Meera	T1.1 Design (4h) T1.2 Registration API (8h) T1.3 KYC integration (10h) T1.4 Sign-up screens (8h) T2.1 Gateway sandbox (4h) T2.2 Top-up API and ledger (8h)

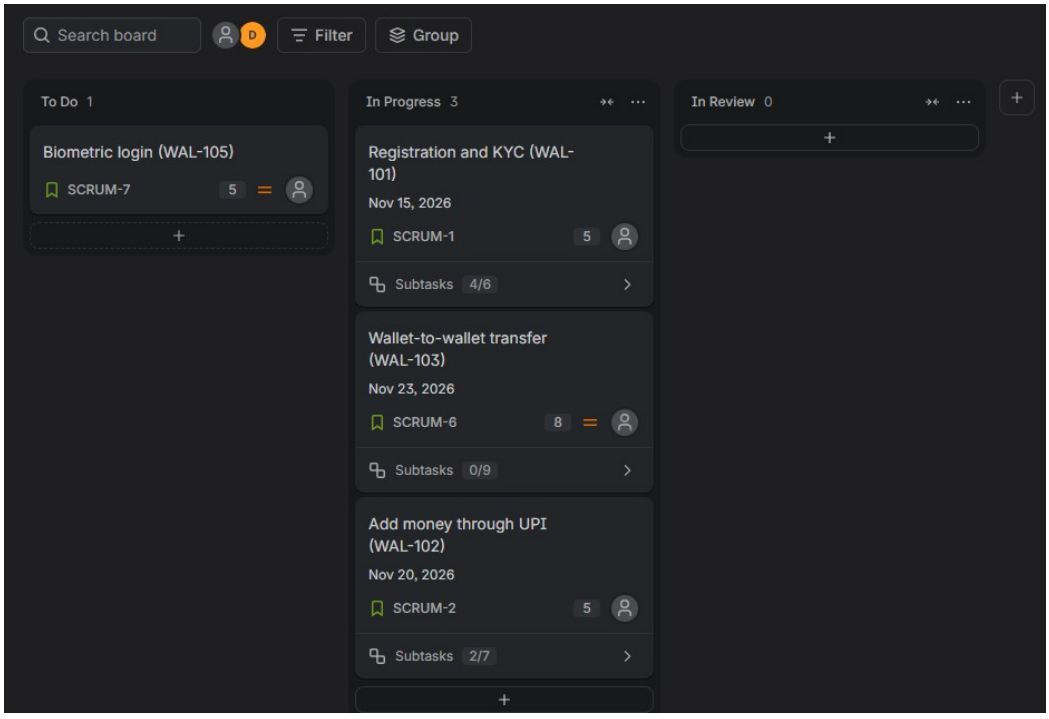


Figure 1: Jira board at the Day 3 stand-up (WAL-101 4/6, WAL-102 2/7, WAL-103 0/9 subtasks done; WAL-105 not started).

Task 3 – Daily stand-up update, Day 3

Team member	Yesterday	Today	Impediment
Priya	Finished the KYC document upload and verification integration (T1.3) and the sign-up screens (T1.4).	Complete tests (T1.5) and raise the pull request for WAL-101 review.	KYC sandbox allows only 5 requests per minute, so integration tests run slowly.
Karthik	Set up the gateway sandbox (T2.1) and built the top-up API and ledger (T2.2).	Build webhook handling for payment success and failure (T2.3).	Sandbox needs a public HTTPS callback URL; waiting for DevOps to expose one.
Meera	Designed atomic debit / credit logic and started the transfer API (T3.1).	Finish T3.1 with row-level locking and start balance validation (T3.2).	Shared staging database is reset nightly for QA, which wipes her test data.

Impediment log

ID	Day	Impediment	Impact	Action and owner	Status
IMP-1	3	KYC sandbox rate limit of 5 requests per minute	T1.5 tests slowed by about 2 h	Mock the KYC service for most tests, keep a few live tests – Priya	Resolved Day 3

ID	Day	Impediment	Impact	Action and owner	Status
IMP-2	3	No public HTTPS callback URL for payment webhooks	T2.3 cannot be tested end to end	Expose a tunnel / staging URL – DevOps, raised by SM	Resolved Day 4
IMP-3	3	Shared staging database reset nightly	Meera loses transfer test data every day	Create a separate dev schema for the team – SM with QA lead	Resolved Day 5
IMP-4	4	Refund rules unclear (full or partial refund, which transactions qualify)	Refund story cannot be estimated reliably	Refinement session with PO and business – PO	Resolved Day 4

Task 4 – Re-prioritising the backlog after the refund-to-wallet request

Added	WAL-111 Refund-to-wallet, estimated at 5 points after a short refinement session. It reuses the ledger built in WAL-102, so the estimate is small.
Removed	WAL-105 Biometric login (5 points) goes back to the product backlog at the top, ready for the next sprint. It supports the Sprint Goal least, because the goal is about adding and moving money, and it has the same size as the new story.
Who decides	**Product Owner:** decides what comes in and what goes out, and the new order of the backlog. **Developers:** estimate the new story and confirm that the swap fits capacity. **Scrum Master:** facilitates the discussion and protects the team from extra scope with nothing removed. **Business sponsor:** is consulted; the request goes through the PO, not directly to developers.
Effect	A 5-for-5 swap keeps the sprint at 23 points. The Sprint Goal does not change, because refund-to-wallet is part of the same money-movement outcome.

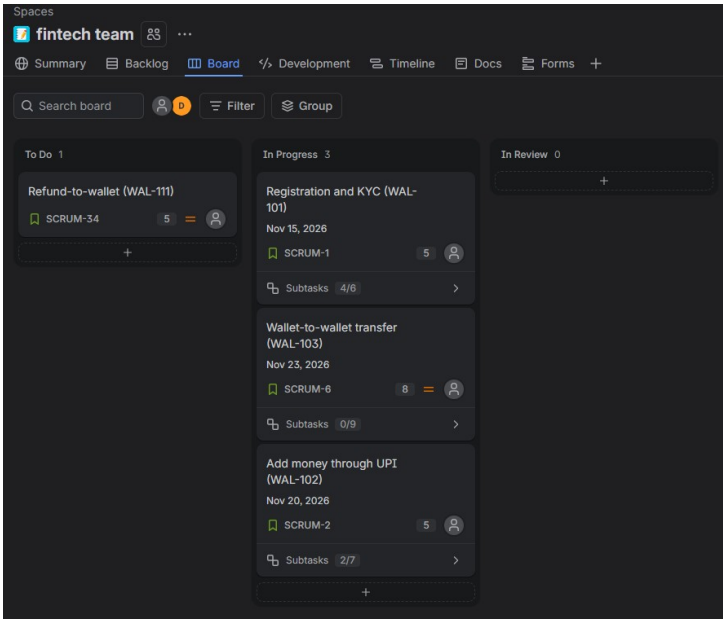


Figure 2: Jira board after the Day 4 change (WAL-111 added, WAL-105 moved to backlog; sprint still 23 points).

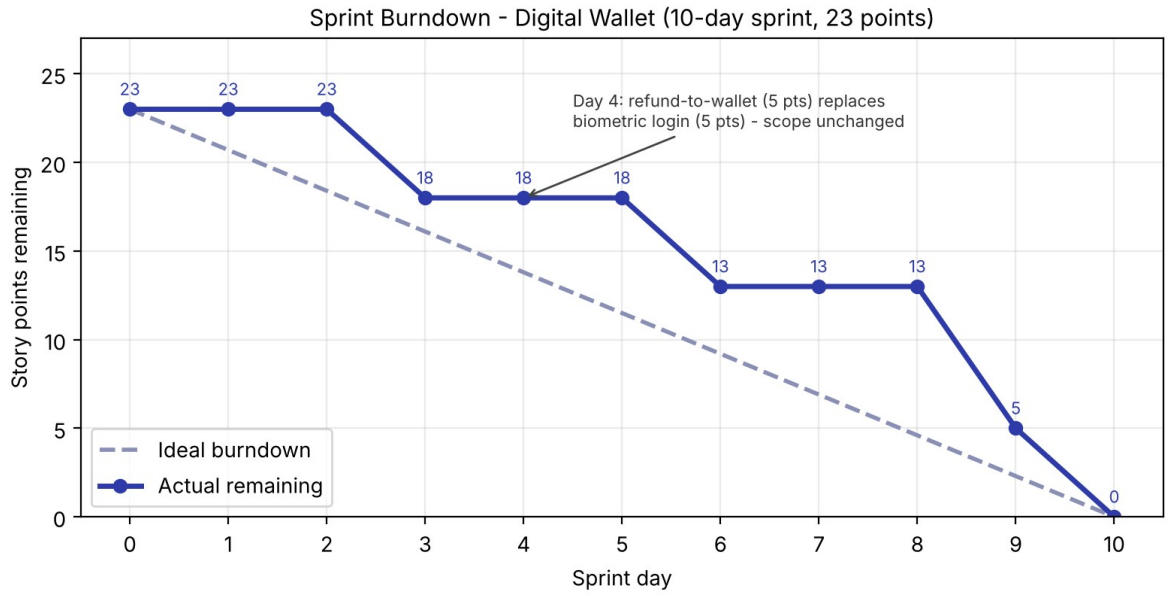
Rank	ID	Story	Points	Status
1	WAL-101	Registration and KYC	5	In sprint
2	WAL-102	Add money through UPI	5	In sprint
3	WAL-103	Wallet-to-wallet transfer	8	In sprint
4	WAL-111	Refund-to-wallet (new)	5	Added to sprint

Rank	ID	Story	Points	Status
5	WAL-105	Biometric login	5	Next sprint, top
6	WAL-104	Transaction history	3	Backlog
7	WAL-106	Low-balance alert	2	Backlog
8	WAL-109	PDF statement	3	Backlog
9–11	WAL-107, 108, 110	Analytics, merchant QR, referrals	21	Backlog

Task 5 – Sprint burndown and interpretation

Points are burned only when a story is done. Day 0 is the start of the sprint with 23 points.

Day	0	1	2	3	4	5	6	7	8	9	10
Completed	–	0	0	5	0	0	5	0	0	8	5
Remaining	23	23	23	18	18	18	13	13	13	5	0
Ideal	23	20.7	18.4	16.1	13.8	11.5	9.2	6.9	4.6	2.3	0



Interpretation

- **Behind the ideal line for most of the sprint.** The actual line sits above the ideal line from Day 3 onwards. The gap is widest on Day 8: 13 points left against 4.6 expected.
- **Staircase shape.** Points only drop when a whole story is done, so the line is flat on Days 1–2, 4–5 and 7–8. This means several stories were open at the same time and work was not finished in small slices.
- **The Day 4 scope change is invisible.** Adding 5 points and removing 5 points nets to zero, so the line has no jump. Adding without removing would have shown a 5-point step up.
- **Zero buffer at the end.** The 8-point transfer story closes on Day 9 and the refund story on Day 10. The Sprint Goal is met, but any delay on the last day would have missed it.
- **Recommendation:** split stories to 3 points or fewer, integrate earlier, and check the burndown at every stand-up so a flat line is noticed within two days.

Task 6 – Retrospective and Definition of Done mapped to the pipeline

Sprint retrospective

What went well	What to improve
• Mid-sprint change handled calmly: PO, developers and SM agreed a 5-for-5 swap within one day. • Stand-ups surfaced three blockers on Day 3 and all were cleared within two days. • The ledger built for WAL-102 was reused, so the refund story stayed small.	• Work finished late: the sprint ended with no buffer and a flat burndown for long stretches. • Environment blockers (sandbox limits, callback URL, shared database) cost developer time. • Refund requirements arrived unclear and needed an extra refinement session.

Action items

#	Action	Owner	Due	Success measure
1	Slice stories to 3 points or fewer and add a Definition of Ready check (clear acceptance criteria) at backlog refinement.	Product Owner with Priya	Before next sprint planning	No story above 5 points; first story done by Day 3
2	Provision a dedicated test environment, a public webhook endpoint and mock services before the sprint starts.	Karthik with DevOps	Day 1 of next sprint	Zero environment impediments logged in the first 3 days

Definition of Done mapped to the build pipeline

Stage	Definition of Done criteria checked	Tooling (example)	Exit gate
Build	Code compiles, lint passes, no critical static-analysis issues, no secrets in code.	Git, Jenkins or GitHub Actions, SonarQube	Pipeline build is green
Test	Unit tests pass with at least 80% coverage on new code; integration and acceptance-criteria tests pass; security scan clean.	JUnit / Jest, API test suite, dependency scan	All tests pass, no high-severity finding
Review	Pull request approved by at least one other developer; deployed to staging and accepted by the PO in a demo.	Pull request review, staging environment	PR approved and PO accepts the story
Merge	Squash-merge to main after all checks are green; documentation and release notes updated; story moved to Done.	Protected main branch, CI merge check	Merged build ready to release

Deliverables checklist

Deliverable	Where in this document
Sprint backlog	Task 1 (selected stories), Task 2 (tasks and sprint board) and Task 4 (re-ordered backlog)
Impediment log	Task 3 (stand-up and impediment log)
Burndown chart	Task 5 (data table, chart and interpretation)
Retrospective actions	Task 6 (retrospective and two action items with owners)
Half-page pipeline mapping	Task 6 (Definition of Done mapped to build, test, review and merge)